推理优化实践手册

> 技术专题整理，不涉及具体厂商与发布事件；数字为工程经验值，换硬件与模型需要重测。

1. 首 token 延迟与每 token 延迟

推理分为预填充与解码两个阶段。预填充把整段输入一次性算完，决定首 token 延迟；解码逐 token 生成， 决定每 token 延迟与总时长。两个阶段的瓶颈不同：预填充受算力限制，解码受显存带宽限制。 优化时必须分开看 P50 与 P95，把两个阶段混在一个平均延迟里，会掩盖真正的瓶颈。

2. 键值缓存与分页注意力

自回归生成会缓存历史 token 的键值对，缓存命中可以避免重复计算。分页注意力把缓存切成固定大小的块， 按需分配，减少显存碎片。工程上要盯三个数：缓存占用比例、缓存命中率、以及序列被截断的比例。 缓存命中率突然下降，通常意味着会话切分或前缀复用的策略被改动过。

3. 连续批处理

连续批处理允许在一条序列生成结束时立刻补入新请求，不必等整批结束。它能显著提升吞吐， 但会拉高单请求的排队时间；高负载下 P99 延迟往往由排队而不是计算决定。压测时必须同时报告 并发数、吞吐与 P95、P99，只报吞吐会把排队成本藏起来。

4. 量化

量化把权重从十六位浮点压到八位或四位整数。权重量化实现简单、收益直接；激活量化更省显存， 但对异常值敏感。上线前必须做一份固定题集的对比：准确率下降超过可接受阈值就回退， 而不是靠主观感觉判断“看起来没变差”。低于四位的激进量化通常只适合演示。

5. 投机解码

投机解码用小模型先草拟若干 token，再由大模型一次性校验，接受的部分直接采用。 它在输出确定性较强的任务上收益明显，在创造性写作上接受率低、收益有限。 选型时要报告接受率与端到端加速比，而不是只讲理论倍数。

6. 压测与容量

压测必须固定输入长度分布，否则缓存命中率会把结果带偏。建议至少覆盖三档输入： 短问短答、长文档问答、多轮对话。每档记录吞吐、P50、P95、P99、显存峰值与降级次数。 容量规划以 P99 为准，不用平均值；平均值好看而尾部卡顿的服务，在真实用户那里仍然是不可用。

7. 常见误区

- 只看平均延迟：尾部才是用户投诉的来源。
- 把重排或检索的耗时算进模型延迟：定位问题会跑偏。

- 压测请求全部命中缓存：得到的是缓存性能，不是推理性能。
- 为了降延迟无限提高单卡并发：排队时间会把收益吃回去。